

Grace Asaad, Taylor Atkinson, TJ Hall, Suraj Srinivasa, Isaiah Deppong, Ini Ogunsanya

Professor Ali Sever

Programming Languages

November 2025

4102 Final Project Report

For this project, the team decided to write a basic HTTP server using Swift. As the group was more familiar with backend work, they wanted to explore using Swift for a similar purpose. This decision was reinforced by the group's curiosity to see how Swift could be utilized outside of its primary use, especially focusing on its use alongside the GNU/Linux platform in more server-side applications and lower-level operations. As safety concerns rise for C/C++ software, companies invest millions of dollars into developing new languages to replace C/C++ with as much control and performance, but with more security features involved.

Swift is a multi-paradigm language that is most famous for its involvement with Apple. It was created for iOS, macOS, watchOS, and tvOS applications to ensure the safety of the products that Apple produces. Swift is also designed to incorporate a variety of programming models and can support traditional object-oriented programming and protocol-oriented programming.

Swift was officially launched by Apple in 2014 and was built into the NeXt platform. Before using Swift, Apple primarily used Objective-C, which is an object-oriented programming language built to add SmallTalk-like features to C. Primary design choices were to eliminate

unsafe features like null pointer dereferencing from systems languages, with its syntax designed to encourage users to write clean and consistent code.

The motivation for the development and design of Swift was to create a language that was efficient, but more importantly, safe. Objective-C had some memory safety issues, which is why Apple software developers felt the need to create a more secure programming language. They were able to utilize their knowledge and experience from creating Objective-C and integrate that into Swift, making sure to account for trends and changes in modern languages during the time of conception.

Data Types:

Elements:

Fundamental Data Types:

Int, Double, Bool, String.

Collection Data Types:

Array, Set, and Dictionary.

Tuples:

A way to store multiple values in a single variable.

Optionals:

nil

Optional Binding()

Structs are used when one wants a class to be passed around by copy rather than by reference in a program. Enums are useful when one wants to have a common type for a group of

related values so instead of modifying the types within a struct or class, one can modify the data in the enum without ever caring about the type.

Var v. Let:

- Swift allows for the var and let keywords to assign variables.
- Var is a true variable, meaning that its value can change at runtime.
- Let's make a constant value that does not need to be known at compile time but can only be assigned once.

It is a combination of many different languages, hailing from Objective-C. One can draw many similarities between any part of Swift and any other given language. For example, it has closures similar to C#, type inference like dynamic languages, and strong typing like Java. It is a very versatile and mixed language, giving a lot of control to the user rather than declaring a strict syntax.

Basic control abstractions:

Swift contains all the normal control abstractions of every other language.

For/In, If/Else, While, Switch, etc.

Handles abstraction:

Type Inferencing:

Allows the compiler to determine the type of a variable at runtime

Type Annotations:

For specific use cases, the developer can also specify which type the variable should have at runtime

If/Let:

In Swift, one can assign a variable optionally, which allows for a variable declaration that may not always return a value, but always returns one.

For Example:

```
if let foo = bar() {  
    foo = "done"  
}  
else {  
    foo = "error"  
}
```

This will make sure no matter whatever happens in bar(), foo will either return done or error.

Function Return Types:

When declaring a function one can use an arrow to explicitly define the return type of a function. Ex: `func sayHi() -> String { return "Hi" }`

One can have different instances of the same method; an example given in the docs is a counter class with different increment methods. One called with no arguments increments the count by 1, one called with an argument specifically an integer it will increment the count by the amount passed in the argument giving flexibility while keeping language concise.

A primary strength of Swift is that it is an open-source programming language. It is more accessible to a wider range of people and relatively easy to learn because of its simple syntax. When creating Swift, the designers prioritized a simple and clean syntax so that it reads more like natural English compared to other programming languages.

Swift offers safety checks such as the elimination of entire classes of unsafe code, which is achieved through predefined checks like arrays being checked for potential overflow, as well as implementing automatic memory management. Swift does not allow the use of nil objects unless it is for specific cases in which it is allowed, which are known as optionals, and forces you to handle them; otherwise, it forces a compile-time error. Because of these features, Swift is far safer to implement compared to C.

Another strength is its fast performance speed. True to its name, Swift is designed for efficiency. Apple reports that it can run up to 2.6 times faster than Objective-C, its predecessor. This performance leads to smoother user experiences and also broadens the scope of what developers can do, enabling them to write more complex and computation-heavy applications without sacrificing responsiveness.

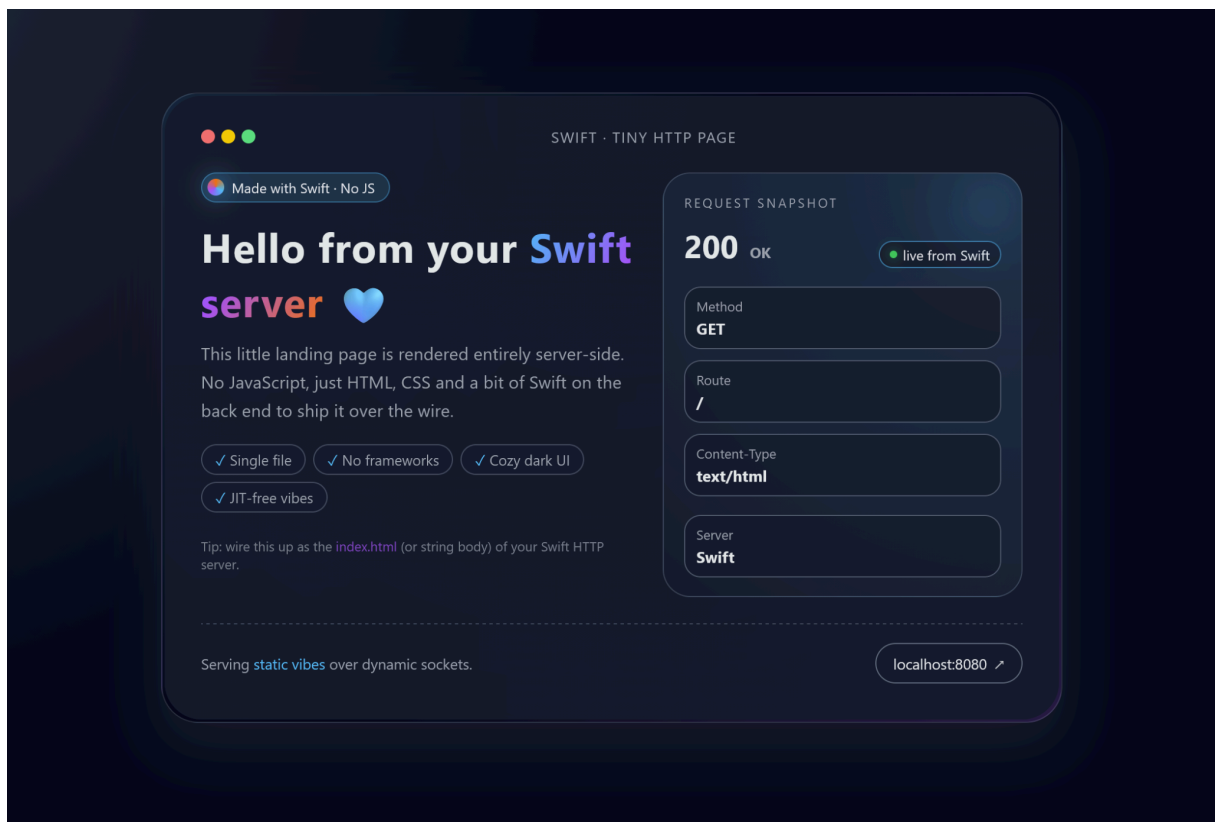
One initial weakness of Swift was that it was a proprietary language, which means that it was owned by one company, but on December 3rd, 2015, Apple released version 2.2 under the Apache License, making it open source and available on Linux; however, it is still a new language and lacks maturity. For example, a problem with languages in their infancy is an underdeveloped package ecosystem, as well as a lack of highly proficient engineers and projects. A great way for a language to quickly develop toolchains and encounter bugs and issues is by having a large, dedicated user base to experiment with the language.

Swift was designed as a general-purpose language, and that it is, while it is not common for lower-level languages, it still handles things surprisingly well with some caveats. Swift is often called a moving target language, meaning that frequently the language undergoes changes that make it behave differently from past versions. For instance, for the team's server, they had to

work with raw pointers, and one can do this in Swift, but they now must use functions to handle the access in memory. Previously, one could simply use pointers like any other language, without needing to understand the Swift API for handling pointers with functions.

Sample run:

```
teej@TeejsLaptop: /mnt/c/Users/tjhal/Desktop/ITCS4102/4102Final/HTTP$ swift run
Building for debugging...
[10/10] Linking http
Build of product 'http' complete! (19.76s)
Server live at http://localhost:8090
```



Outside its primary role in building applications for devices in Apple's ecosystem, the team realized that Swift's capabilities extended to server-side development, which the group tapped into to explore deeply. Ted Kremenek, Apple's Director of Languages and Runtime, even

suggested that Swift has the potential to be a suitable replacement (or successor) to long established system languages like C and C++. With this perspective in mind, the group's problem and project revolved around how to determine how effectively Swift could perform server-side activities. The group also evaluated whether its modern language features, performance advantages, and security focused approach would benefit other development areas.

The Proposed method was to write a basic HTTP server interoperating with C/C++ to highlight its interoperability and performance as a general-purpose programming language for many high and low-level applications. Simple HTTP servers are a great example to highlight this, as they typically require some use of the TCP/IP stack, socket interfacing, and occasionally multi-threading/parallel programming. This demonstrates the low-level uses well and is a real-world application, rather than simply a performance test. It also aids in showing how ergonomic the language is at utilizing external libraries, kernel level networking, memory management, and string manipulation.

Swift built-in testing packages provide a platform for test-driven development with some pretty extensible usage, primarily with the built-in package manager known as SPM (Swift Package Manager). This manager/framework is called XCTest and it is designed for unit, integration, and UI testing of Swift applications. Unit tests can be designed to validate the most critical features of the group's system, ensuring they work correctly and the system can be loaded and run adequately.

Experiments:

Swift claims to have a compiler that is optimized for performance.[3] Some experiments to test are handling speed with its competitor, C/C++.

In conclusion, Apple saw a need for a more secure programming environment. Their already existing system (objective-C) was unable to meet all their security and syntax needs. They decided to create exactly what they were looking for, by pulling inspiration from objective-C, Python, Haskell, C# and CLU by combining their best, most helpful and important features into one unified modern, and secure syntax.

This group decided to work with Swift because they wanted to test it outside of its primary purpose. Through experimentation the team realized that Swift was able to adapt for server-side applications.

Works Cited

Apple. “Swift - Apple Developer.” *Apple.com*, 2025, developer.apple.com/swift/.

“Apple Developer Documentation.” *Developer.apple.com*,
developer.apple.com/documentation/swift/.

Cooper, Dylan. “The Era of Swift 6 Has Arrived! It’s the Best Choice over C++.” *Medium*,
CodeElevation, 2 July 2024,
medium.com/codeelevation/the-era-of-swift-6-has-arrived-its-the-best-choice-over-c-4963ac8246be. Accessed 24 Nov. 2025

“Documentation.” *Docs.swift.org*,
docs.swift.org/swift-book/documentation/the-swift-programming-language/aboutswift/.

GeeksforGeeks. “Swift Programming Language.” *GeeksforGeeks*, 8 Mar. 2018,
www.geeksforgeeks.org/swift/swift-programming-language. Accessed 24 Nov. 2025.

“Introduction.” *Developer.apple.com*,
[developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjectiveC/Introduction/i
ntroObjectiveC.html](https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjectiveC/Introduction/introObjectiveC.html).

“Testing a Swift Package. How I Added Unit Tests and Github... | by Marco Longobardi |
Medium.” *Medium*, 17 July 2025,
medium.com/@marco.longobardi997/testing-a-swift-package-08be3d166b75